

Universidad

Arquitectura de Computadoras

Diseño e implementación de una UART para interfaz con ALU

Trabajo Práctico N.º 2

Estudiantes: Javier Siñuka y David Trujillo

Fecha: 30 de septiembre de 2026

Índice

1. Introducción	1
2. Objetivos	1
3. Arquitectura general	1
4. Formato UART y generación de baudios	2
5. Máquinas de estados finitos	2
5.1. Receptor y transmisor UART	3
5.2. Controlador UART–ALU	3
6. Buffers y protocolo de aplicación	3
7. Integración en la Basys 3	4
8. Simulación y verificación	4
9. Conclusiones	5

Resumen

Este informe documenta el sistema UART implementado en Verilog para comunicar una PC con la ALU de ocho bits del Trabajo Práctico N.º 1. El diseño integra un generador de baudios, recepción y transmisión UART, buffers de un byte y máquinas de estados finitos que organizan el flujo de datos. La PC envía tres bytes binarios —operando A, operando B y código de operación— y la FPGA devuelve un byte con el resultado. Se describen el formato 8N1, el sobremuestreo por dieciséis, las FSM, la integración en Basys 3 y las pruebas realizadas.

1. Introducción

Una UART (*Universal Asynchronous Receiver and Transmitter*) permite intercambiar datos serie sin que los extremos compartan un reloj. En este TP se usa como interfaz entre una computadora y la ALU: los switches del trabajo anterior se reemplazan por un enlace serie que recibe los operandos y devuelve el resultado. La consigna propone precisamente un receptor UART, un transmisor, un generador de baudios y un circuito de interfaz que conecta la comunicación serie con la ALU.

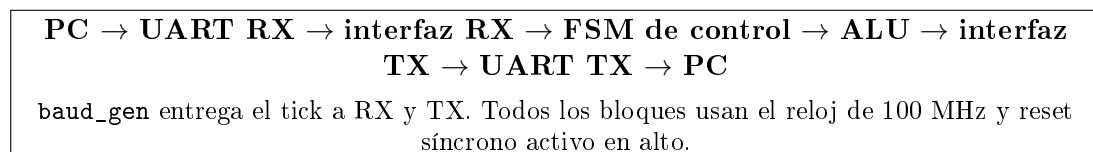
Las FSM modelan circuitos secuenciales cuyo comportamiento depende del estado actual y de eventos de entrada. Se aplican en RX, TX y en el controlador que reúne los tres bytes de un comando. Esta separación permite analizar por independiente la temporización de cada bit, el almacenamiento temporal y el cálculo combinacional de la ALU.

2. Objetivos

- Implementar recepción y transmisión UART 8N1: ocho datos, sin paridad y un bit de parada.
- Generar un tick dieciséis veces más rápido que el baud rate para muestrear cerca del centro de cada bit.
- Aplicar FSM para RX, TX y el flujo de datos UART–ALU.
- Integrar la ALU de ocho bits mediante un protocolo binario de tres bytes y transmitir el resultado por UART.
- Verificar módulos e integración con testbenches antes de sintetizar.

3. Arquitectura general

El recorrido de un comando es el siguiente:



El módulo superior `uart_top` instancia `baud_gen`, `uart_rx`, `interface_circuit` y `uart_tx`. El controlador `interface_circuit` contiene los buffers RX/TX y la ALU heredada de TP1. Así, los bloques UART sólo serializan o deserializan, mientras que la interfaz conserva bytes y el controlador define cuándo consumirlos.

Cuadro 1: Responsabilidad de los módulos.

Módulo	Función
<code>baud_gen</code>	Divide el reloj de placa y produce el pulso de sobremuestreo.
<code>uart_rx</code>	Sincroniza RX, detecta inicio, reconstruye el byte y verifica parada.
<code>uart_tx</code>	Emite inicio, ocho bits LSB primero y parada; informa ocupación.
<code>interface_rx</code>	Buffer de un byte con lectura, estado vacío y error/desbordamiento.
<code>interface_circuit</code>	FSM que carga A, B y operación; instancia la ALU y ordena transmitir.
<code>interface_tx</code>	Conserva un resultado y genera un único pulso de inicio para TX.
<code>uart_top</code>	Integra reloj, reset, USB-UART y LEDs de la Basys 3.

4. Formato UART y generación de baudios

La línea UART está en uno durante reposo. Una trama 8N1 comienza con un cero de inicio, sigue con los ocho bits de datos desde el menos significativo hasta el más significativo y termina con un uno de parada. Por lo tanto, cada byte ocupa diez períodos de bit.

RX y TX usan sobremuestreo $16\times$. Para el reloj de 100 MHz y la configuración de placa de 9600 baudios, el divisor entero es:

$$N = \left\lfloor \frac{100\,000\,000}{9600 \times 16} \right\rfloor = 651. \quad (1)$$

El contador de `baud_gen` cuenta de 0 a 650 y emite un tick de un ciclo. Esto equivale a aproximadamente 153 609 Hz, frente a los 153 600 Hz ideales; el error de división entera es despreciable.

Listing 1: Contador del generador de tick

```

1 localparam CNTR_MAX = CLOCK_FREQ / (BAUD_RATE * OVERSAMPLE) - 1;
2 always @(posedge i_clock) begin
3     if (i_reset) r_counter <= 0;
4     else if (r_counter == CNTR_MAX) r_counter <= 0;
5     else r_counter <= r_counter + 1;
6 end

```

Luego de detectar un posible inicio, el receptor lo confirma al tick 7, en el centro de ese bit. Las muestras de los datos también se toman al tick 7 de cada intervalo de dieciséis, tal como indica la metodología de la consigna.

5. Máquinas de estados finitos

Los módulos usan un bloque combinacional para `r_next_state` y otro sincronizado por `i_clock` para `r_state`. Los estados se declaran con `localparam` y se codifican en binario: cada FSM

tiene cuatro estados. La rama **default** vuelve al estado inicial y provee recuperación frente a un estado inválido.

5.1. Receptor y transmisor UART

uart_rx recorre **IDLE**, **START**, **DATA** y **STOP**. En **IDLE** espera la línea baja; en **START** descarta un falso inicio si la línea volvió a uno a mitad del bit. En **DATA** desplaza las ocho muestras al registro **r_shift**; en **STOP** publica **o_data** y genera **o_valid** por un ciclo. Si la parada está en cero, también activa **o_error**.

Como RX es asíncrona, se aplica un sincronizador de dos flip-flops antes de la FSM:

Listing 2: Sincronización de la entrada serial

```
1 always @(posedge i_clock) begin
2     if (i_reset) r_rx_sync <= 2'b11;
3     else r_rx_sync <= {r_rx_sync[0], i_rx};
4 end
```

uart_tx utiliza los mismos cuatro estados. Al aceptar **i_start** en **IDLE**, registra **i_data**, pone la salida en cero y permanece dieciséis ticks en **START**. Durante **DATA** transmite **r_shift[0]** y desplaza el registro a la derecha al finalizar cada bit; después transmite el uno de parada. La señal **o_busy** permanece activa durante toda la trama, por lo que un nuevo inicio no altera el byte ya registrado.

5.2. Controlador UART–ALU

La FSM de **interface_circuit** implementa la secuencia:

READ_A → READ_B → READ_OP → WRITE_RESULT → READ_A.

En cada estado **READ_*** espera que RX deje de estar vacío y activa **w_rd**; en el flanco siguiente almacena el byte en el registro de A, B u operación. Después de cargar la operación, **WRITE_RESULT** mantiene las entradas de la ALU estables durante un ciclo completo. Cuando TX está libre activa **w_wr**, la interfaz TX captura el resultado y la FSM vuelve a esperar un nuevo A.

Listing 3: Transición al finalizar una operación

```
1 WRITE_RESULT: begin
2     // La ALU has a full cycle to settle after r_op.
3     if (w_wr)
4         r_next_state = READ_A;
5 end
```

Un error de recepción devuelve el controlador a **READ_A** y limpia los registros parciales. La organización de las salidas de control por estado se corresponde con una FSM de Moore, mientras que las condiciones de avance usan **rx_empty**, **tx_full** y las señales de error.

6. Buffers y protocolo de aplicación

interface_rx conserva un byte. Acepta un dato válido si está vacío o si se lo lee en el mismo flanco: la lectura consume el byte previo y se guarda el recién llegado. Si llega un byte válido con el buffer ocupado y sin lectura, se detecta un desbordamiento, se vacía el registro y se genera **o_error**. Los bytes con error de parada también se descartan.

`interface_tx` mantiene el resultado hasta que `uart_tx` lo acepta y termina de enviarlo. Su FSM recorre `IDLE`, `WAIT_TX`, `WAIT_BUSY` y `WAIT_DONE`. Esperar que `busy` suba antes de continuar garantiza un solo pulso `start`; mientras no está en `IDLE`, `o_tx_full` evita sobrescribir el dato pendiente.

La consigna muestra las señales de interfaz, pero no determina el orden de los bytes de comando. Se eligió el protocolo binario de la Tabla 2.

Cuadro 2: Protocolo UART–ALU implementado.

Byte	Interpretación
1	Operando A de ocho bits.
2	Operando B de ocho bits; en desplazamientos se usan sus bits [2:0].
3	Código de operación: se usan los bits [5:0] y se ignoran [7:6].
Respuesta	Resultado de la ALU, un byte.

Por ejemplo, transmitir 12 34 20 en hexadecimal devuelve 46: el código 20 selecciona la suma y $0x12 + 0x34 = 0x46$. No se envían los caracteres ASCII. Los códigos heredados son ADD 20, SUB 22, AND 24, OR 25, XOR 26, SRA 03, SRL 02 y NOR 27; un código no implementado devuelve cero.

El protocolo no tiene cabecera, delimitador ni *timeout*: un comando parcial espera los bytes restantes y, tras perder un byte, no puede realinearse automáticamente. Para recuperarse se detiene el emisor, se aplica reset y se reenvían los tres bytes. Dado que los buffers son de una posición, la PC debe esperar la respuesta antes del comando siguiente.

7. Integración en la Basys 3

`uart_top` se parametriza para 100 MHz y 9600 baudios. El archivo `Basys3_UART.xdc` asigna el reloj a W5, el botón central de reset a U18 y el puente USB–UART a B18 (entrada RX de FPGA) y A18 (salida TX). LD0–LD7 muestran el último resultado aceptado por la interfaz TX; LD8 indica reset, LD9 que TX está ocupado y LD10 retiene un error hasta el siguiente reset.

El botón y RX son asíncronos al reloj de placa. El TOP sincroniza el botón con dos flip-flops y `uart_rx` hace lo mismo con la línea serie. El XDC marca dichas cadenas como `ASYNC_REG` y restringe los caminos asíncronos sólo hasta la primera etapa. La inicialización `2'b11` del sincronizador de reset deja el sistema inicialmente en reset aunque el botón esté suelto.

Para operar con la PC se configura 9600 baudios, 8 datos, sin paridad, un bit de parada y sin control de flujo. En Vivado se agregan los ocho archivos `.v` como *Design Sources*, `uart_top` como módulo superior y únicamente `Basys3_UART.xdc` como constraints del TP.

8. Simulación y verificación

Se escribieron testbenches unitarios para el generador de baudios, RX, TX y ambas interfaces, más pruebas del controlador, del enlace UART completo y del TOP. Para reducir el tiempo de simulación, las pruebas de integración elevan el baud rate mediante parámetros; el diseño sintetizable conserva 9600.

Cuadro 3: Cobertura de testbenches.

Testbench	Comprobaciones
<code>tb_uart_rx</code>	Bytes A5, 00, FF; falso inicio, parada inválida y tramas consecutivas.
<code>tb_uart_tx</code>	Inicio, datos, parada, dato estable, intento de inicio ocupado y envíos sucesivos.
<code>tb_interface_rx</code>	Lectura/recepción simultánea, desbordamiento, errores y reset.
<code>tb_interface_tx</code>	Pulso único, ocupación, conservación del dato y reset pendiente.
<code>tb_interface_circuit</code>	Ocho operaciones, códigos inválidos, comandos parciales, TX ocupado, errores y reset.
<code>tb_uart_interface</code>	Doce comandos y respuestas transmitidos como tramas seriales.
<code>tb_uart_top</code>	Trece respuestas, reset inicial y por botón, LEDs y recuperación.

Las ejecuciones realizadas aprobaron RX (26 comprobaciones), TX (22), las dos interfaces, el controlador (22 resultados y cuatro errores esperados), la integración UART (12 comandos/-respuestas) y el TOP (13 respuestas, reset y LEDs). `tb_baud_gen` confirmó 651 ciclos entre ticks posteriores, pero reportó una diferencia de un ciclo en su aserción del primer tick: lo observa en el ciclo 652 y espera 651. Esa discrepancia debe revisarse en la fase de liberación de reset o en la convención de conteo del testbench; por transparencia, no se considera esa prueba completamente aprobada.

9. Conclusiones

Se desarrolló una UART completa para operar la ALU desde una PC. La separación entre serialización, buffers, FSM de control y lógica combinacional facilita la comprensión y la verificación del diseño. El sobremuestreo por dieciséis ubica las muestras en el centro de los bits, y las FSM establecen una secuencia segura para recibir A, B y la operación antes de transmitir el resultado.

La integración con Basys 3 incorpora sincronización de señales asíncronas, indicadores de diagnóstico y las restricciones físicas necesarias. Las pruebas validan el funcionamiento de RX, TX, interfaces, controlador y TOP. Como pendiente concreto queda ajustar o documentar la expectativa del primer tick en `tb_baud_gen`, de forma que toda la batería de simulación quede sin discrepancias.